Práctica. Triaje de alertas con Wazuh

Nombre y apellidos: _____

Miércoles 4 de noviembre de 2026 · IC · 4 horas · trabajo individual

Ya tenéis Wazuh funcionando en vuestro equipo y habéis visto llegar las primeras alertas. Hoy vamos a hacer lo que hace un analista de SOC de primer nivel nada más sentarse en su puesto: mirar una cola de alertas y decidir cuáles importan, cuáles son ruido y por cuál empieza. A eso se le llama **triaje**.

La sesión tiene dos partes. En la primera provocas tú unos eventos controlados en tu equipo y los buscas en tu Wazuh. Así sabes con certeza qué ha pasado y puedes comprobar si Wazuh lo cuenta bien. En la segunda te doy una cola de alertas de una empresa ficticia, con un incidente escondido entre el ruido, y tienes que encontrarlo, ordenarlo y explicarlo.

1. Qué vas a aprender hoy

1. Comprobar el estado del manager y de tus agentes desde la línea de comandos.
2. Provocar eventos de seguridad controlados y relacionarlos con la alerta que generan.
3. Leer una alerta de Wazuh: regla, nivel, descripción, agente, registro original y técnica MITRE.
4. Filtrar y ordenar alertas con `jq` o con el visor que te doy.
5. Clasificar alertas en verdadero positivo, falso positivo y benigno esperado.
6. Priorizar y reconstruir la cronología de un incidente.

2. Lo que necesitas

Tu Wazuh dockerizado (el de la práctica 0) y al menos un agente Linux conectado. Si tu Wazuh no funciona, puedes hacer igualmente toda la segunda parte: no depende de él.

Cada uno tiene su propio Wazuh en su equipo. No hay un servidor común, así que no busques las alertas de un compañero en el tuyo ni le pidas su contraseña.

Herramientas: `docker compose`, `jq` (si no lo tienes, `sudo apt install jq`) o, si no puedes instalarlo, `python3` con el visor `ver_alertas.py` del material.

Descarga el ZIP del laboratorio de la web del curso (IC, Prácticas) y descomprímelo:

```
mkdir -p ~/ic
cd ~/ic
unzip ~/Descargas/IC-2026-11-04-triaje-LAB.zip
ls IC-2026-11-04-triaje-LAB
```

3. Comprueba que tu SOC está vivo (20 minutos)

En el equipo donde tienes Wazuh:

```
cd ~/wazuh-docker/single-node
docker compose ps
```

Tienen que aparecer `wazuh.manager`, `wazuh.indexer` y `wazuh.dashboard` en estado `running` (o `Up`). Si alguno está parado o reiniciándose, no sigas probando cosas al azar: abre el checklist de troubleshooting de Wazuh que tenéis en la web y revisa por orden recursos, contenedores, puertos, manager y agente.

Ahora pregunta al manager qué agentes conoce:

```
docker compose exec wazuh.manager /var/ossec/bin/agent_control -l
```

Verás una línea por agente con su ID, nombre, IP y estado. El primero, con ID `000` y la palabra `server`, es el propio manager. Tu agente Linux tiene que aparecer como `Active`. Apunta su nombre: lo vas a usar para filtrar.

Si tu agente sale como `Disconnected` o `Never connected`, en el equipo del agente revisa:

```
sudo systemctl status wazuh-agent --no-pager
sudo grep -n "<address>" /var/ossec/etc/ossec.conf
sudo tail -n 30 /var/ossec/logs/ossec.log
```

La dirección del manager tiene que ser la IP real del equipo donde está Wazuh, no `127.0.0.1` si el agente está en otra máquina. En el `ossec.log` busca líneas con `ERROR` o `Unable to connect`. Si después de revisarlo sigue sin conectar, pasa a la segunda parte (apartado 6) y documenta lo que has comprobado: eso también se evalúa.

4. Un poco de teoría antes de tocar nada (25 minutos)

Cada alerta de Wazuh nace de una **regla** que se ha cumplido. La regla tiene un número (`rule.id`), un **nivel** de 0 a 15 y una descripción. Cuanto más alto es el nivel, más grave considera Wazuh el suceso, aunque el nivel lo pone la regla, no tú. Una regla de nivel 5 puede formar parte de un ataque serio y una de nivel 7 puede ser una actualización del sistema.

Algunas reglas son **de correlación**: no se disparan por una línea de registro, sino por un patrón de varias. Por ejemplo, ocho fallos de contraseña desde la misma IP en poco tiempo generan una alerta de fuerza bruta de nivel 10. Y si después de esos fallos llega un acceso correcto desde la misma IP, se dispara otra de nivel 12. La primera dice "alguien está probando contraseñas"; la segunda dice "y probablemente ha acertado".

Al hacer triaje, cada alerta (o cada grupo de alertas del mismo suceso) acaba en una de estas casillas:

- **Verdadero positivo**: ha pasado algo malo y la alerta lo refleja.
- **Falso positivo**: la regla se ha disparado, pero lo que describe no ha ocurrido o está mal interpretado.
- **Benigno esperado**: ha ocurrido de verdad, pero es una actividad legítima (una actualización, un administrador trabajando). Muchos SOC lo llaman también "verdadero positivo benigno".
- **No concluyente**: con lo que tienes no puedes decidir. Hay que decir qué dato falta.

Para **priorizar** no basta con el nivel. Pregúntate tres cosas: qué gravedad tiene lo que describe, qué importancia tiene el equipo afectado (no es lo mismo el servidor web que un PC del aula) y si hay indicios de que el ataque ha tenido éxito.

Un detalle que da muchos problemas: el manager guarda la hora de las alertas en **UTC**. En noviembre, en la España peninsular, vamos una hora por delante de UTC. Si generas un evento a las 10:15, la alerta aparecerá como `09:15` con `+0000` al final.

5. Primera parte: provoca eventos y búscalos (75 minutos)

Todo esto se hace **en tu equipo Linux con agente**, con tu propio usuario. No lo hagas en equipos de otros ni en servidores del centro.

5.1. Apunta la hora de inicio

```
date
```

Escríbela en la plantilla. Cada vez que hagas una de las pruebas siguientes, apunta también la hora. Sin horas no hay cronología.

5.2. Tres fallos de sudo

```
sudo -k
sudo true
```

La primera orden olvida la contraseña que sudo tenía guardada; la segunda te la pide. Escribe tres veces una contraseña **incorrecta** a propósito. Al tercer fallo sudo se rinde con `3 incorrect password attempts`.

5.3. Crea y borra un usuario de prueba

```
sudo useradd -M -s /usr/sbin/nologin ic-prueba
sudo userdel ic-prueba
```

`-M` hace que no se le cree carpeta personal y `-s /usr/sbin/nologin` impide que pueda iniciar sesión. Lo borramos justo después: solo queremos la huella en los registros.

5.4. Cambios en la carpeta vigilada

Si en la práctica de FIM configuraste la vigilancia de `/opt/ic-lab/monitor`:

```
echo "triaje 4 nov" > /opt/ic-lab/monitor/triaje.txt
echo "segunda línea" >> /opt/ic-lab/monitor/triaje.txt
rm /opt/ic-lab/monitor/triaje.txt
```

Deja unos segundos entre una orden y otra. Si no hiciste aquella práctica, sáltate este punto y apúntalo como "no configurado".

5.5. Una marca con logger

```
logger "IC-LAB triaje inicio $(hostname)"
```

Esta orden escribe un mensaje en el registro del sistema. Fíjate en si genera alerta o no y piensa por qué.

5.6. Recoge tus alertas

Espera uno o dos minutos y, en el equipo de Wazuh, copia las últimas alertas a un fichero:

```
cd ~/wazuh-docker/single-node
docker compose exec -T wazuh.manager \
    tail -n 500 /var/ossec/logs/alerts/alerts.json > ~/ic/mis-alertas.jsonl
```

`-T` hace que Docker no prepare una terminal interactiva, que es lo que necesitas cuando rediriges la salida a un fichero. Ahora míralas filtrando por tu agente (cambia `NOMBRE_AGENTE` por el que apuntaste en el apartado 3):

```
cd ~/ic/IC-2026-11-04-triaje-LAB
python3 ver_alertas.py ~/ic/mis-alertas.jsonl --agente NOMBRE_AGENTE
```

Vas a ver muchas alertas que no has provocado tú, sobre todo del agente `000` con reglas de SCA (la comprobación de configuración segura que Wazuh hace solo). Es normal y es justo lo que tenéis que aprender a filtrar.

Con lo que has hecho deberían aparecer, entre otras, alertas parecidas a estas. Los números de regla son los de Wazuh 4.14.8; si tu versión es otra, pueden variar:

Lo que hiciste	Regla	Nivel	Descripción
Fallo de contraseña en sudo	5503	5	PAM: User login failed.
Tres fallos seguidos de sudo	5404	10	Three failed attempts to run sudo
Crear el usuario	5902	8	New user added to the system.
Borrar el usuario	5903	3	Group (or user) deleted from the system.
Crear, modificar o borrar en la carpeta vigilada	554, 550, 553	5, 7, 7	File added / Integrity checksum changed / File deleted

Busca también las mismas alertas en el Dashboard: en el menú, Threat Hunting, pestaña Events. Filtra por `agent.name` y ajusta el intervalo de tiempo. Haz una captura de una de ellas abierta, con el registro original (`full_log`) visible.

5.7. Si algo no aparece

No escribas que Wazuh "no funciona" sin comprobarlo. Revisa por este orden:

1. La hora: ¿has convertido a UTC? ¿El intervalo del Dashboard incluye el momento de la prueba?
2. El agente: ¿sigue `Active` en `agent_control -l`?
3. La fuente: ¿el agente lee el registro donde se escribe el evento? En el equipo del agente, `sudo grep -n "<location>" /var/ossec/etc/ossec.conf` muestra qué ficheros lee. Comprueba si existe `/var/log/auth.log` con `ls -l /var/log/auth.log`.
4. El evento: ¿se ha escrito de verdad? `sudo grep ic-prueba /var/log/auth.log` debería mostrar las líneas de `useradd` y `userdel`.
5. La regla: ¿ese evento tiene una regla con nivel suficiente? Es lo que le pasa al `logger`: el mensaje llega, pero ninguna regla lo convierte en alerta.

Si no consigues que aparezca, documenta qué has revisado y qué resultado has obtenido en cada paso. Un "no aparece, he comprobado esto, esto y esto" vale mucho más que una captura copiada.

6. Segunda parte: triaje de una cola sintética (70 minutos)

El fichero `alertas-SINTETICAS-2026-11-02.jsonl` tiene 27 alertas de una empresa inventada con tres equipos: un servidor web (`srv-web01`), el PC de secretaría (`pc-secretaria`) y un PC de aula (`pc-aula07`). Todo es inventado, pero las reglas, niveles y descripciones son las que pone Wazuh a esos registros. Las alertas están **desordenadas**, como cuando llegan de varias fuentes.

6.1. Ordénalas y obtén una primera visión

Con `jq`:

```
F=alertas-SINTETICAS-2026-11-02.jsonl
jq -s -r 'sort_by(.timestamp)[] | [.timestamp[11:19], .agent.name,
  .rule.id, .rule.level, .rule.description] | @tsv' $F
```

`-s` lee todas las líneas como una sola lista, `sort_by(.timestamp)` las ordena por hora y `@tsv` las saca en columnas separadas por tabuladores. Sin `jq`, lo mismo con el visor:

```
python3 ver_alertas.py alertas-SINTETICAS-2026-11-02.jsonl
```

6.2. Cuenta y agrupa

Cuántas alertas hay de cada regla:

```
jq -s -r 'group_by(.rule.id)[] | [.[0].rule.id, length,
  .[0].rule.description] | @tsv' $F
```

De qué IP vienen los accesos:

```
jq -r '.data.srcip // empty' $F | sort | uniq -c | sort -rn
```

`// empty` descarta las alertas que no tienen IP de origen. `uniq -c` cuenta las repeticiones y `sort -rn` pone arriba la que más aparece.

Las alertas más graves, con su registro original:

```
jq -c 'select(.rule.level >= 10) | {hora: .timestamp,
  regla: .rule.id, log: .full_log}' $F
```

Con el visor: `python3 ver_alertas.py $F --min-nivel 10` o `--ip LA_IP`.

Los cambios en ficheros (FIM):

```
jq -r 'select(.syscheck) | [.timestamp[11:19], .agent.name,
  .syscheck.event, .syscheck.path] | @tsv' $F
```

6.3. Clasifica

Rellena la tabla de `plantilla-triaje.md`. No clasifiques alerta por alerta: agrupa las que forman parte del mismo suceso (por ejemplo, todas las de una misma IP en unos minutos) y clasifica el grupo. Para cada grupo justifica la decisión con datos de la alerta: no vale "parece normal".

Algunas pistas para pensar, sin darte la respuesta:

- Una actualización de paquetes cambia ficheros del sistema. ¿Qué hora tiene y qué ficheros toca?
- Equivocarse una vez con la contraseña de sudo le pasa a cualquiera. ¿Qué pasa justo después?
- Las IP `10.20.0.x` son de la red interna. ¿Y las demás?
- Fíjate en qué usuarios se intentan en los primeros minutos del ataque y compáralos con el usuario que se intenta después.
- Un fichero `.php` nuevo dentro de una carpeta `uploads` de un servidor web es algo que conviene mirar dos veces.

6.4. Reconstruye la cronología

Del grupo que consideres más grave, escribe la cronología paso a paso: hora, qué ocurre y qué alerta lo demuestra. Separa lo que **sabes** (está en una alerta) de lo que **supones** (lo deduces, pero no tienes la prueba). Por ejemplo, que se ha creado un usuario lo sabes; para qué lo quería el atacante, lo supones.

6.5. Prioriza y propón

Ordena los grupos de más a menos urgente y justifica el orden. Para el primero, escribe las tres primeras cosas que comprobarías o pedirías (por ejemplo, qué registros revisar, qué preguntar al responsable del servidor o qué evidencia guardar antes de tocar nada). No ejecutes ninguna contención: eso lo trabajaremos más adelante.

7. Elige tu itinerario (35 minutos)

Recuperación

Si tu Wazuh no está completo, termina primero la práctica 0 (instalación), la práctica 1 (despliegue del SOC) y la de FIM. Las tienes en la web, en IC, Prácticas. Lo mínimo para hoy es tener un agente `Active` y haber hecho la segunda parte de esta hoja con la cola sintética.

Ampliación

Wazuh trae una herramienta para probar qué regla dispara una línea de registro sin tener que generarla de verdad. Ábrela dentro de tu manager:

```
docker compose exec wazuh.manager /var/ossec/bin/wazuh-logtest
```

Copia de la cola sintética, una a una y en orden, las líneas `full_log` de los fallos de contraseña de `tecnico` y después la del acceso aceptado. Fíjate en qué regla sale con cada línea y en qué momento cambia. Explica por qué la misma línea `Accepted password` da una regla distinta si antes ha habido fallos desde la misma IP. Para salir, pulsa `Ctrl+C`.

Después busca en la web de MITRE ATT&CK las técnicas que aparecen en `rule.mitre.id` de las alertas más graves y explica con tus palabras qué describe cada una.

8. Preguntas

1. ¿Qué diferencia hay entre el nivel de una alerta y su prioridad en el triaje?
2. ¿Por qué el mensaje de `logger` no ha generado ninguna alerta? ¿Significa eso que Wazuh no lo ha recibido?
3. De la cola sintética, ¿qué grupo de alertas has clasificado como benigno esperado y qué dato te ha permitido decidirlo?
4. ¿Qué alerta te parece la más importante de toda la cola y por qué no es la de nivel más alto, o por qué sí?
5. ¿Qué información te falta para poder afirmar que el servidor web está comprometido?

9. Qué tienes que entregar

Una carpeta `NN-IC-triaje` comprimida en ZIP con:

1. `plantilla-triaje.md` rellena: tus alertas reales, la clasificación de la cola sintética, la cronología y las siguientes comprobaciones.
2. Una captura del Dashboard con una de tus alertas abierta y el `full_log` visible. Antes de hacerla, comprueba que no sale ninguna contraseña.
3. Tu fichero `mis-alertas.jsonl` (si tu Wazuh funciona) o, si no, el registro de lo que has comprobado según el apartado 5.7.
4. Las respuestas a las preguntas del apartado 8 (en la propia plantilla o en un `respuestas.md`).

Si no terminas, entrega lo que tengas e indica en qué punto te has quedado.

10. Antes de entregar, comprueba que

1. Cada alerta real de tu tabla tiene al lado la acción que la provocó y la hora local y UTC.
2. Cada grupo de la cola sintética tiene clasificación y una justificación con datos.
3. La cronología distingue lo que sabes de lo que supones.
4. Has borrado el usuario `ic-prueba` (`id ic-prueba` debe decir que no existe).
5. Ni las capturas ni los ficheros llevan contraseñas, tokens ni datos de compañeros.

11. Seguridad

Todas las pruebas de hoy son sobre tu propio equipo y tu propio Wazuh. No lances intentos de acceso contra equipos de compañeros ni del centro, aunque sea "para ver si sale la alerta". En un trabajo real, generar eventos en un sistema sin autorización es un incidente, no una prueba.

Los datos de la cola sintética son inventados. El rango 203.0.113.0/24 está reservado para documentación y no pertenece a nadie.

Referencias

- Wazuh, reglas y niveles: https://documentation.wazuh.com/current/user-manual/ruleset/index.html
- Wazuh, wazuh-logtest: https://documentation.wazuh.com/current/user-manual/reference/tools/wazuh-logtest.html
- jq, manual: https://jqlang.org/manual/
- MITRE ATT&CK: https://attack.mitre.org/